Recommendation Systems for Personalized Content Discovery

Technical Report — Netflix Prize Dataset

Team Members: Suyash Verma , Shubham Saini **Institution:** IIT Roorkee
Submission: Cultural Council Open Projects 2026

---

1. Problem Understanding

Modern streaming platforms serve hundreds of millions of users, making personalized content discovery a core engineering challenge. The goal of this project is to build a recommendation system that can learn from historical user-movie interactions and generate meaningful, personalized movie recommendations.

We worked with the Netflix Prize Dataset — one of the most well-studied benchmarks in recommendation system research. The dataset was released as part of a 2006 competition to improve Netflix's recommendation accuracy by at least 10% over their internal algorithm (Cinematch). This gives our work a clear, real-world grounding.

Our objective was not just to minimize prediction error, but to build a system capable of ranking relevant content effectively — which is ultimately what matters for user experience.

---

2. Exploratory Data Analysis

2.1 Dataset Overview

We worked with a sampled subset of 500,000 ratings from the full dataset due to computational constraints. The full dataset contains 100.48 million ratings from 480,189 users across 17,770 movies.

Metric	Value
Ratings (sample)	500,000
Unique Users	~48,000
Unique Movies	~9,800
Rating Scale	1–5
Date Range	Oct 1998 – Dec 2005
Matrix Sparsity	~99.89%

2.2 Rating Distribution

The rating distribution is notably left-skewed, with a strong bias toward higher ratings (3 and 4 being the most common). Very few users gave 1-star ratings, which suggests that users who watch a movie are already

somewhat likely to enjoy it — they chose to watch it in the first place. The mean rating in our sample was approximately 3.60.

This has an important implication: a naive baseline that always predicts 3.6 would actually achieve a reasonably low RMSE, which means our models need to do meaningfully better than this to be useful.

2.3 User Activity (The Long Tail Problem)

User activity follows a heavy long-tail distribution. A small fraction of users (~5%) contribute more than 50% of all ratings. The median user has rated only about 20 movies, while the top users have rated thousands.

This poses a classic challenge: our models work well for "power users" who have rich interaction histories, but struggle with sparse users. This is what the cold-start problem refers to.

2.4 Movie Popularity

Movie popularity is even more skewed. The top 1% of movies (by rating count) account for a disproportionately large share of all ratings. Blockbuster titles from the early 2000s (the dataset's time range) dominate. Unknown or niche films may have only a handful of ratings, making it hard to reliably estimate their quality.

2.5 Temporal Patterns

Rating activity peaked around 2004–2005, which aligns with the Netflix Prize competition's active period. Interestingly, average rating scores remained relatively stable across years (around 3.5–3.7), suggesting no significant "rating inflation" over time.

2.6 Key Takeaways from EDA

- The data is extremely sparse (~99.9%), making matrix factorization approaches well-suited.
- A global mean baseline (predict 3.60 for everything) is a strong lower bound to beat.
- Long-tail distributions in both users and movies require careful handling.
- Temporal information could be used for more sophisticated models but is out of scope for our MVP.

---

3. Methodology

We implemented two recommendation approaches and compared them head-to-head.

3.1 Approach 1: User-Based Collaborative Filtering (User-CF)

Core Idea: If user A and user B have rated many of the same movies similarly in the past, they probably have similar tastes. So if user A loved Movie X and user B hasn't seen it, we recommend it to user B.

Steps:

1. Build a user × movie rating matrix (sparse, since most entries are 0/missing).
2. Mean-center each user's ratings (to remove the "harsh rater vs. generous rater" bias).
3. Compute pairwise cosine similarity between all users.
4. For a target (user, movie) pair: find the top-k most similar users who have rated that movie, compute a weighted average of their ratings (weighted by similarity), add back the target user's mean.

5. Clip the prediction to [1, 5].

Key parameter: k = 30 nearest neighbors.

Limitation: Scalability. Computing the full user-user similarity matrix is $O(n^2)$ in users. For our 48K users, this requires ~9 billion pair computations. We handled this by operating on the sparse matrix and only computing similarities on demand, but it's still the slowest part of our pipeline.

3.2 Approach 2: SVD — Matrix Factorization

Core Idea: Factorize the rating matrix R into two lower-dimensional matrices: R ≈ P × Q^T, where P contains user "taste vectors" and Q contains movie "attribute vectors". The dot product P_u · Q_m gives the predicted rating. These latent vectors are learned by gradient descent, minimizing prediction error on known ratings.

We use Simon Funk's biased SVD (as popularized during the Netflix Prize competition itself) via the Surprise library. Biased SVD adds user and movie bias terms: `predicted(u, m) = μ + b_u + b_m + P_u · Q_m`, where μ is the global mean, b_u is user bias (some users just rate higher), and b_m is movie bias (some movies just get rated higher).

Hyperparameters:

- Latent factors: 100
- Epochs: 20
- Learning rate: 0.005
- Regularization: 0.02

Why SVD over User-CF?
SVD generalizes better to sparse data, is much faster at inference time, and handles the global mean + bias structure naturally. User-CF is more interpretable ("these neighbors liked it") but doesn't scale.

3.3 Train-Test Split

We used a random 80/20 train-test split. We acknowledge that a temporal split (train on earlier ratings, test on later ones) would more accurately simulate real deployment, since it prevents the model from seeing "future" knowledge about users. However, for this scope, random splitting is standard and produces reliable RMSE estimates.

Only test entries where both the user and movie appeared in the training set are kept for evaluation (we can't predict for completely unseen users/items in collaborative filtering).

---

4. Model Evaluation

4.1 RMSE — Rating Prediction Accuracy

RMSE (Root Mean Squared Error) measures how far off our predicted ratings are from the actual ratings. A lower score is better.

Model	RMSE	MAE
Global Mean Baseline	~1.12	~0.93

Model	RMSE	MAE
User-Based CF (k=30)	~0.96	~0.75
SVD (100 factors)	~0.88	~0.69

SVD outperforms User-CF on rating prediction. Both significantly beat the global mean baseline, validating that personalization adds real value.

4.2 MAP@10 — Recommendation Ranking Quality

MAP@10 (Mean Average Precision at 10) measures whether the movies we actually recommend (the Top-10 list) include movies the user genuinely liked. A movie is "relevant" if the user's actual rating is ≥ 3.5 (per the problem statement).

Model	MAP@10
Random Recommendations	~0.05
SVD	~0.18

A MAP@10 of 0.18 means that, on average, about 18% of the Top-10 recommendations were movies the user actually liked. This is significantly better than random (5%) and reflects the value of our personalized ranking.

4.3 Discussion: RMSE vs MAP@10

An important observation: a model with lower RMSE doesn't always have higher MAP@10. RMSE optimizes for average prediction accuracy, but MAP@10 cares about whether you can correctly rank the truly great movies at the top. These are related but distinct objectives. A production system would likely optimize specifically for ranking (using pairwise or listwise loss functions), which is beyond our current scope.

---

5. Recommendation Examples

Success Case:
A user who had given 5 stars to Gladiator, Braveheart, and The Lord of the Rings received Top-10 recommendations that included several high-rated epic/adventure titles they hadn't seen — consistent with their demonstrated preference for the genre.

Failure Case:
For a user with very few ratings (< 5 in train), predictions were unreliable and defaulted near the global mean. This is expected behavior — the cold-start problem is a known limitation of collaborative filtering.

Key Observation:
Popular movies appear frequently in recommendations across users, even when not personalized. This "popularity bias" is a known problem in CF systems — they tend to over-recommend popular items. Techniques like re-ranking with diversity metrics could address this.

---

6. Key Insights

1. **Sparsity is the core challenge.** With 99.9% of entries missing, any approach must handle sparse data gracefully. SVD does this better than memory-based methods like User-CF.
2. **Bias terms matter enormously.** Adding user and movie biases to SVD (biased SVD) improves RMSE by ~8% over unbiased SVD. Some users just consistently rate 0.5 stars higher than average — capturing this is important.
3. **RMSE ≠ good recommendations.** We observed that minimizing RMSE does not guarantee useful rankings. A system could achieve low RMSE by predicting 3.6 for everything, but would fail at MAP@10.
4. **The long tail is underserved.** Popular movies perform well in evaluation simply because they appear in more users' histories. Niche/arthouse films are systematically harder to recommend well.
5. **Scalability requires architectural choices.** On a laptop with 500K ratings, SVD trains in ~3 minutes. On the full 100M dataset, this would scale to ~10 hours without GPU acceleration or distributed training.

---

7. Cold Start Strategy

One of the fundamental challenges of collaborative filtering is the "Cold Start" problem — making recommendations for new users who have no interaction history, or new movies that have no ratings.

In our deployed dashboard, we implemented a fallback heuristic:

- **New Users:** If a user ID has no historical ratings in the dataset, the system defaults to a **Popularity Baseline**. It recommends the highest-rated globally popular movies.
- **Sparse Users:** For users with very few ratings, the SVD model's bias terms (user bias and movie bias) dominate the dot product, gracefully degrading the prediction toward the global mean adjusted by item popularity.
- **Content-Based Fallback:** Our hybrid engine utilizes the release decade from the movie metadata to inject content-based signal, which helps surface relevant items even when the collaborative signal is weak.

---

8. Future Improvements

- **Temporal dynamics:** Weight recent ratings more heavily (users' tastes change over time).
- **Neural Collaborative Filtering:** Use deep learning to learn non-linear user-item interactions.
- **Deep Hybrid Approaches:** Combine CF with rich metadata (synopses, cast) using embeddings.

---

9. Repository Structure

```
netflix-recommender/
├── data/           # Dataset files (movie_titles.csv, combined_data_1.txt)
├── outputs/        # Generated EDA charts
├── presentation/   # Presentation.pptx
├── report/         # This technical report
├── app.py          # Monolithic Streamlit Dashboard (Model + Hybrid Engine + UI)
```

```
├── README.md
└── requirements.txt
```